Übungsblatt 3: Speicherzugriff & Kontrollfluss

Mikrocomputertechnik 1

Zielsetzung

In diesem Übungsblatt brechen wir aus der isolierten 32-Register-Werkbank aus. Sie lernen, wie Sie Daten über Load- und Store-Befehle zwischen der CPU und dem Arbeitsspeicher (RAM) hin- und herschieben. Zudem programmieren Sie Ausbrüche aus dem linearen Programmablauf, um If/Else-Bedingungen und for-Schleifen in hardware-nahem Assembler umzusetzen.

Aufgabe 1: Pointer-Mathematik und Alignment

Ein Integer-Array beginnt im RAM an der Basisadresse (Base-Pointer), die im Register `s1` gespeichert ist.

Array-Index und Offset

In C greifen Sie auf das vierte Element mit `array[3]` zu. Wie lautet der exakte Assembler-Befehl, um dieses Element in das Register `t0` zu laden? Bedenken Sie die 4-Byte-Schrittweite von 32-Bit-Integern.

Alignment Fault

Was passiert physikalisch in der CPU, wenn Sie versuchen, ein 32-Bit-Wort von der Adresse `0x01` (z. B. mit dem Offset `1(s1)`) zu laden?

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Aufgabe 2: Teilzugriffe und Sign-Extension

Sie haben ein einzelnes Byte mit dem Hex-Wert `0x8F` (binär: `1000 1111`) im RAM liegen. Sie laden dieses Byte mit dem Befehl `lb t0, 0(s1)` in ein leeres 32-Bit-Register.

Sign-Extension nach `lb`

Wie sieht das vollständige 32-Bit-Bitmuster im Register `t0` nach diesem Befehl aus (in Hexadezimal)? Erklären Sie kurz, warum die CPU die linken 24 Bits genau so auffüllt.

Unsigned Load

Welchen Befehl müssten Sie nutzen, wenn das Byte einen reinen RGB-Farbwert (ohne Vorzeichen) darstellt und Sie die Vorzeichenerweiterung verhindern wollen?

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Aufgabe 3: If/Else in Assembler

Übersetzen Sie das folgende C-Konstrukt strikt in RISC-V-Assembler. Halten Sie sich an das 4-Schritte-Paradigma aus der Vorlesung (Bedingung invertieren, If-Block, rettender Jump, Else-Block).

Nehmen Sie an, dass Variable `a` im Register `s0` und Variable `b` in `s1` liegt. Das Ergebnis soll in `t0` gespeichert werden.

```
if (a == b) {
    t0 = 1;
} else {
    t0 = 2;
}
```

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)

Aufgabe 4: Venus-Programmierung — Array-Iteration

Wir setzen die Labor-Aufgabe aus der Vorlesung im Venus-Simulator um. Schreiben Sie eine for-Schleife, die alle Elemente eines Arrays aufsummiert.

Aufgabenstellung

1. Legen Sie im `.data`-Segment ein Array mit den Werten 10, 20, 30, 40, 50 an (Direktive `.word`).
2. Wechseln Sie in das `.text`-Segment und laden Sie die Basisadresse des Arrays mit `la s1, myArray`.
3. Initialisieren Sie einen Schleifenzähler (`t0 = 0`), ein Limit (`t1 = 5`) und eine Summen-Variable (`a0 = 0`).
4. Programmieren Sie die Schleife:
   - Prüfen Sie am Start die Abbruchbedingung (`bge`).
   - Laden Sie den Wert von der aktuellen Adresse in `s1` (`lw`) und addieren Sie ihn zu `a0`.
   - Erhöhen Sie den Array-Pointer `s1` um 4 (nächstes Element).
   - Erhöhen Sie den Schleifenzähler `t0` um 1.
   - Springen Sie zurück zum Schleifenanfang.
5. Verifizieren Sie das Endergebnis (150 bzw. `0x96`) in `a0` mit dem Step-Debugger.

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)